HORUS — Pre-Study for the Project Assignment Form (PAF)

COMP 4910/4920 Senior Design Project, Fall 2026 – Spring 2027 · Advisor: Assoc. Prof. Dr. Arman Savran

29 September 2026

Purpose. This document answers the advisor's four pre-PAF questions. The one-page PAF (`HORUS-PAF.docx`) is compiled from it. Numbers marked *measured* come from our simulation prototype's logs (`sim/logs/`, `runs/`). Numbers marked *plan* are targets.

1. Why is this problem worth solving?

1.1 Problem

Frontier vision-language models such as Claude and GPT can follow an open-ended instruction like *"find the person in the red shirt, check whether they are hurt, then come back"*. No drone autopilot can do this today. Three things stop us from simply connecting the model to the drone:

- **The model is too slow for flight control.** Frontier models run in the cloud and answer in seconds, while a drone has to react within tens of milliseconds. *Measured:* one S6 "find the red shirt" mission with Claude as commander took 70–98 s of wall time and 8–13 tool calls, costing about US$0.50 per run.
- **The link is unreliable.** Wi-Fi or cellular links add 0.5–3 s of delay, lose packets and drop out, especially indoors and in disaster areas. By the time an answer arrives, the image it refers to is out of date and the target has moved.
- **Unsafe commands.** A model that sends velocity commands can fly into a wall. *Measured (S3):* with ArduPilot proximity avoidance off, an executor-issued `goto` toward a wall crossed the drone's body envelope and crash-disarmed. With avoidance on, the same command stopped at 1.29–1.44 m clearance.

Research question: how can a slow, remote AI reliably direct a fast physical system over a delayed or intermittent link while staying general across tasks and safe?

1.2 Who benefits

- **Search and rescue (SAR) in GPS-denied spaces.** Examples are collapsed or smoke-filled buildings, corridors and basements. After events such as the February 2023 Kahramanmaraş earthquakes, the bottleneck is trained pilots and their attention, not drones. An operator who says *what* to look for, instead of flying stick by stick, can supervise the search while doing other work.
- **Other supervised inspection and monitoring tasks** with the same pattern: open-ended goals, specific targets and an unreliable link.
- **Robotics research.** The same pattern — a slow model in the cloud and a fast executor onboard — applies to any robot controlled by a cloud model.

1.3 Why now

MCP (Model Context Protocol) gives language models a standard, authenticated tool interface. Small onboard accelerators (4 TOPS) can run a detector, tracker and re-identification network in real time. ArduPilot provides proven stabilisation, proximity avoidance, geofencing and return-to-home (RTH). Each piece exists; nobody has combined them into a system that is safe and tolerant of latency (Section 2).

2. Are there similar projects? What is our difference?

2.1 Short answer

Yes, many projects cover *parts* of HORUS: language-controlled drones, aerial vision-language navigation, MCP flight tools, visual grounding and commercial follow-me drones. In the works we reviewed, these parts are split across different projects. None documents the full HORUS combination:

1. The cloud model picks a **persistent onboard tracker ID** drawn on the frame (set-of-mark), not pixel coordinates.
2. An **onboard executor compensates for 0–3 s of stale observations** and runs only bounded skills.
3. ArduPilot remains an **independent safety layer** for avoidance, geofence and RTH.
4. We **evaluate** latency tolerance and safety directly: a latency sweep and a cloud-only baseline.

We claim the **integration and its latency/safety evaluation** as new, not any single component.

2.2 Closest prior work (verified sources)

Work	What it does	Real drone?	Latency handling	Independent safety layer	Gap HORUS addresses
TypeFly (Yale, 2023–24)	GPT-4 writes a program in a small task language (MiniSpec); a local interpreter plus YOLOv8 executes it on a DJI Tello	Yes (Tello)	Shortens time to first action; no compensation for stale frames	Restricted language; no geofence or autopilot layer	Closest prior art. Plans once; no persistent IDs, no delay recovery, no safety stack
ChatGPT for Robotics (Microsoft, 2023)	ChatGPT generates code that calls robot functions; Tello and AirSim demos	Yes (Tello) + sim	None	A human reviews the code	No visual identity, no executor, no latency handling
CognitiveDrone / -R1 (Skoltech, 2025)	Vision-language-action (VLA) model maps FPV image + text to actions; slow reasoning module	No (Gazebo, SITL)	Two model rates; no network delay	None reported	Simulation only; model too heavy to run on the drone

Work	What it does	Real drone?	Latency handling	Independent safety layer	Gap HORUS addresses
SPINE (UPenn, 2025)	Language missions grounded in a semantic map; plans validated before execution	Yes (mostly ground vehicles, one UAV demo)	Tests a distilled onboard LLM for availability	Plan validator	Map and waypoint level; no target track IDs or delay compensation
Say-REAPEx (UniBw Munich, 2024)	LLM + POMDP online SAR planner with YOLOv8	No (UE5/AirSim)	Fewer LLM calls	Feasibility checks	Simulation-only SAR planning; no re-ID, no real flight
UAV-VLA (Skoltech, 2025)	GPT + Molmo generate waypoint plans from satellite images	No	Offline	None	Static plan generation, not closed-loop tracking
EchoPilot (GitHub, 2025)	Voice → local LLM → MCP tools → MAVSDK/PX4	No (SITL)	Local model	Plan review before takeoff	No vision loop, no tracking
ardupilot-mcp (GitHub, 2026)	Safety-gated ArduPilot MCP server; actuation off by default	No (SITL)	—	Yes: gating	Good safety reference; no perception or executor
LLM-controlled-drone (GitHub, 2026)	Local LLM + YOLOv8 → PX4 setpoints via ROS 2	No (Gazebo)	README reports 5–15 s CPU inference	Limits only in the prompt	No stable IDs, no latency recovery, no deterministic safety

Techniques we build on (not competitors): Set-of-Mark prompting, Grounding DINO, the dual-system control idea from Figure Helix and NVIDIA GR00T N1, and aerial person re-ID (AG-VPReID, CVPR 2025). Aerial navigation benchmarks such as OpenFly, CityNav and OpenUAV are simulation datasets, not deployed control systems.

2.3 Commercial products

Skydio Autonomy and DJI ActiveTrack 360° offer excellent onboard avoidance and subject following. However, the task is fixed ("follow this subject"), there is no open-ended language goal, and there is no way for an external AI to direct them. HORUS is complementary: it adds a task-general supervisory layer on an open autopilot.

2.4 Earlier Yaşar COMP senior projects

Project (2025–26)	Summary	Difference from HORUS
IUAV – Intelligent UAV (GBYF finalist)	Fully autonomous air-to-air mission: YOLOv11 + DeepStream target locking, QR dive, no-fly-zone avoidance, SITL + Gazebo	One fixed, pre-programmed mission; simulation only; no language or AI commander; military-style task
HAWKEYE	YOLOv11 + visual servoing to chase another drone	One fixed task ("follow the target drone"); no semantic target choice, no latency or supervisory layer

Both confirm that the department's ArduPilot + Gazebo tooling works. HORUS reuses that toolchain but answers a different question: **task-general, AI-directed, delay-tolerant, civilian SAR**, validated on real hardware.

2.5 Our difference in one table

Property	Typical LLM-drone work	Fixed-task drones (DJI, Skydio, HAWKEYE, IUAV)	HORUS
Task generality	High, but unsafe or offline	Low, one task	**High, through bounded skills**
Grounding	Pixels, code, waypoints	Built-in class	**Onboard tracker ID picked by the cloud model (set-of-mark)**
Delayed or lost link	Not handled	Not needed	**Compensated (frame buffer, fast-forward, re-ID); hold/RTH on expiry**
Safety	In the prompt or none	Onboard	**Deterministic: clamps + ArduPilot avoidance/geofence/RTH, independent of the AI**
Evaluation	Demo	Demo	**Latency sweep 0–3 s and cloud-only baseline, N ≥ 20 trials per condition**

3. Are all concrete goals defined? What are they?

Yes. Scope has three tiers. **The must-have core alone answers the research question**; stretch goals are added only once the core flies reliably.

3.1 Must-have goals and measurable success criteria

#	Goal	Success criterion (plan)
G1	**Three-tier loop**: cloud commander → authenticated MCP tool server → onboard executor → ArduPilot	All skills return task IDs; commander observes progress through events, without polling
G2	**Set-of-mark grounding**: detector + tracker with persistent IDs; numbered frame and candidate table sent to the commander	"Find the person in the red shirt": correct target in **≥ 80 % of 20 sim trials**; the report names the right location
G3	**Tracking and re-identification** through occlusion	"Follow that person": re-acquires after a 2 s occlusion in **≥ 80 %** of trials
G4	**Latency compensation** (frame ring buffer, fast-forward tracking, embedding re-match, slow-down while waiting)	Compensation beats no compensation in success rate and wrong-target rate at **≤ 2 s** added delay
G5	**Deterministic safety**: speed, distance, altitude and geofence clamps; command TTL; hold-by-default; ArduPilot proximity avoidance	Injected unsafe commands blocked or clamped in **100 %** of trials; **zero** AI-caused collisions
G6	**GPS-free indoor flight and return home**: optical flow + rangefinder into EKF3, AprilTag precision landing	Link cut → hold → RTH → lands within a set distance of the tag centre in **every** trial
G7	**Operator console** (web): live marked video, candidate table, mission chat, event log, HOLD/LAND/RTH buttons	Operator can run all core scenarios and emergency actions from the browser
G8	**Real hardware**: quadcopter with prop guards, onboard computer, ToF ring, optical flow, camera	Core scenarios G2, G3, G6 repeated on the real drone in a netted indoor area
G9	**Experiments**: latency sweep	N ≥ 20 per condition in sim; smaller

#	Goal	Success criterion (plan)
	(0/0.5/1/2/3 s × ID mode / box with and without compensation) and **cloud-only vs. HORUS**	confirming set on hardware; results in the final report

3.2 Stretch goals

- **Recognition ladder step 3:** the agent triggers YOLOv8n fine-tuning (`train_specialist`), auto-labelled data, a quality gate, and loading in shadow mode.
- **Disconnected autonomy:** signed mission package, store-and-forward, return to the last area with good link, sync on reconnect.
- **Ground-station edge tier:** grounding model on a laptop GPU on the LAN.
- **One thermal-camera module.**

3.3 Explicitly out of scope

- DroneCAN module auto-discovery
- An onboard (local) LLM
- Visual-inertial odometry (VIO)
- Multi-drone operation
- Outdoor GPS flight on hardware (outdoor worlds exist in simulation only)
- Any weaponised or covert-surveillance use

4. What will the Requirements Specification (RSD) roughly contain?

This follows the COMP 4910 RSD template: requirements list, actors and use cases, non-functional requirements, glossary. RSD v1.0 (textual) is due 30 Oct 2026; v2.0 (UML) is due 20 Nov 2026.

4.1 Actors

- **Operator:** a human SAR team member.
- **Cloud commander:** an AI agent, external system.
- **Onboard executor.**
- **Flight controller:** ArduPilot, external system.
- **Grounding service:** optional, external.
- **Safety pilot:** holds the RC override.

4.2 Use-case groups (draft)

Group	Use cases
UC1 Mission management	Start mission from text · Ask operator on ambiguity · Report result · Reset mission · Replay scripted mission (YAML)
UC2 Perception and grounding	Get marked frame · List candidates · Set open vocabulary · Point by description (fallback) · Set target profile (reference crops)
UC3 Skills	Take off · Go to pose · Approach ID · Follow ID · Orbit ID · Scan area · Wait for task / events · Cancel task
UC4 Safety and emergency	Hold · Land · Return home · Command TTL expiry · Link-loss escalation · Geofence/speed clamp · Proximity stop · RC override

Group	Use cases
UC5 Navigation without GPS	Optical-flow/rangefinder state estimation · AprilTag precision landing
UC6 Latency handling	Buffer frames · Fast-forward a stale box · Re-match by embedding · Slow down while a query is pending
UC7 Evaluation	Run latency sweep · Run cloud-only baseline · Collect ground truth · Produce trial report
UC8 Administration and security	Configure limits (strict config) · Manage tokens · Encrypted logs · Privacy: delete mission profile

4.3 Non-functional requirements (draft)

- **Safety:** the AI never sends raw motor or velocity commands; every speed, distance and geofence is enforced onboard. Emergency tools bypass the task queue. The core must fly safely with zero cloud and zero optional modules. Priority order: safety > failsafe/RTH > executor > cloud.
- **Performance:**
  - Executor control loop 10–20 Hz.
  - Perception frame rate capped to what the onboard accelerator can sustain.
  - The commander is event-driven, at most about 1 Hz.
  - Every observation carries a frame ID, timestamp and pose.
- **Security:**
  - Bearer-token authentication on every MCP request.
  - The listener binds to loopback only and is reached through an encrypted tunnel.
  - Detector labels and autopilot text are passed to the model as quoted, untrusted data (prompt-injection defence).
- **Privacy (KVKK/GDPR):**
  - Person profiles exist only for a lawful SAR purpose and are scoped to one mission.
  - Profiles are deleted after the mission; logs are encrypted.
  - A match is only a candidate until a human confirms it.
- **Portability:** the same ArduPilot firmware and the same software run in SITL and on the drone; the commander backend is swappable (Claude Code / Anthropic / OpenAI-compatible).
- **Reproducibility:** every experiment is a scripted mission with a JSON result; link delay is emulated with `tc netem` on the commander link only, never on MAVLink.

5. Can we deliver this in two semesters with our team?

5.1 Effort budget

Per the COMP 4910 rules, each student gives about 186 h per semester (23.25 man-days). A four-person team therefore has **≈ 744 man-hours per semester and ≈ 1 488 man-hours (186 man-days) across 4910 + 4920**.

5.2 Risk already reduced: working simulation prototype (pre-study)

Before the PAF, we built a working simulation prototype. It uses ArduPilot SITL + Gazebo Harmonic, an indoor room-and-corridor world with animated people, and outdoor forest and urban worlds. The code base is about 11 k lines of Python in `horus/`, about 9 k lines of simulation and experiment tooling, and about 6.8 k lines of tests. Milestone results (*measured*):

Milestone	Result
S1 MAVLink control	2 × 2 m square in Guided mode; corner error 0.04–0.32 m
S2 MCP server + console	`state` / `goto` / `hold` / `rth` from the console; skills return task IDs
S3 Safety proof	Avoidance on: `goto` into a wall stops at 1.29–1.44 m clearance (2 scenarios). Avoidance off (control run): crash
S4 GPS-free + landing	Pad moved 0.5 m from home: AprilTag landing error 1.5–2.9 cm vs. 49.5–51.4 cm with plain RTL (3 + 3 trials)
S5 Follow through occlusion	5 / 5 runs passed; occlusion 2.0–2.2 s; same identity kept
S6 AI commander loop	“Find the person in the red shirt”: 3 / 3 correct (Claude over MCP), landed ≈ 3 cm from pad, US$0.47–0.54 per run
S7 Latency sweep	Harness works; first 23 trials: ID mode 10 / 10, box + compensation 7 / 8, box without compensation 2 / 5; full sweep pending

Therefore the two semesters are **not** spent on proving feasibility from scratch. They go to:

- real hardware,
- the full experimental evaluation,
- the operator console,
- the stretch features,
- the engineering documentation the course requires (RSD, DSD, product manual, final report).

5.3 What we design and build vs. what we use ready-made

Area	We design and develop	Ready-made technology we integrate
Flight and safety	Executor state machine, bounded skills, velocity-only pose controller, clamps, TTL/link-loss logic, geofence projection, precision-landing controller, ToF → MAVLink bridge	ArduPilot (stabilisation, EKF3, proximity avoidance, RTL), pymavlink
Perception	Set-of-mark renderer, tracker with ID preservation, frame/pose ring buffer, fast-forward and embedding re-match, attribute extraction, recognition ladder (embedding match, tiny classifier head)	YOLOv8n (ONNX), OSNet x0.25 re-ID, OpenCV/GStreamer, AprilTag, Grounding DINO / OWLv2 (fallback)
AI layer	MCP tool API design, commander session and event pump, safety prompt, emergency bypass	Claude / GPT (frontier models), Python MCP SDK
Operator	Web operator console	Browser, HTTP/SSE
Test bench	Indoor and procedural outdoor worlds, civilian-actor drivers, ground-truth analysers, latency-sweep and campaign harness	Gazebo Harmonic, ArduPilot SITL, `tc netem`, Docker

Area	We design and develop	Ready-made technology we integrate
Hardware	Frame integration, sensor ring layout, wiring, parameter tuning, TIDL model compilation	T3 Gemstone O1 (TI AM67A, 4 TOPS) or Pixhawk + Raspberry Pi fallback, VL53L1X ToF, optical-flow sensor, camera

We do not train large models or write a flight controller. Our engineering effort goes where the research question is: the executor, grounding, latency compensation, safety and evaluation.

5.4 Work breakdown (4 members; names to be assigned)

Member	Area	Main deliverables
A	Flight, safety and hardware	Drone build, ArduPilot on hardware, ToF/flow/EKF tuning, safety tests, AprilTag landing on hardware
B	Perception and recognition	TIDL compile of detector and re-ID, tracker robustness on drone-view footage, recognition ladder, (stretch) `train_specialist`
C	Executor, latency compensation and experiments	Skills, compensation, full latency sweep, cloud-only baseline, statistics and plots
D	AI commander, MCP, operator console and security	Web console, commander prompts and model comparison, authentication, tunnel, KVKK procedures
All	Documentation and management	RSD v1/v2, DSD, weekly reports, effort logs, final report, presentation, poster

5.5 Timeline aligned with course deadlines

When	Course deliverable	Engineering work (plan)
Fall 2026 (COMP 4910)		
W1–3 (→ 9 Oct)	**PAF**	Pre-study, literature check, hardware bill of materials and order
W4–6 (→ 30 Oct)	**RSD v1.0** (text)	Full S7 latency sweep in sim (300 trials); cloud-only baseline harness
W7–9 (→ 20 Nov)	**RSD v2.0** (UML)	Web console draft (UI demo to advisor); hardware assembly; props-off bench tests
W10–12 (→ 11 Dec)	**DSD v1.0**	Tethered/low-altitude hover with flow + ToF; TIDL compile of YOLOv8n + OSNet
W13–14 (19–25 Dec)	**Presentation, Final Report, PEVAL**	**Hardware go/no-go:** stable indoor hover on Gemstone, or switch to Pixhawk + Raspberry Pi
Spring 2027 (COMP 4920)		
W1–4	Detailed DSD	Indoor hardware flights of core scenarios (find, follow, link-cut RTH)
W5–9	Implementation and testing	Stretch goals in priority order (mission package → `train_specialist` → edge tier); hardware confirmation trials
W10–12	Product Manual	Final experiments; results analysis
W13–14	Final Report (BS thesis), demo, poster	Demo video; GBYF application (Apr–May)

5.6 Hardware, budget and infrastructure

- **Drone:** 5–7” frame with prop guards; T3 Gemstone O1 onboard computer (4× A53 cores for Linux, 2× R5F cores for ArduPilot, 4 TOPS accelerator); 4–6× VL53L1X ToF sensors; optical flow + downward rangefinder; CSI camera.
    - Estimated bill of materials ≈ **US$785–1 190**, or ≈ US$510–915 if the board is provided (design document v2).
    - Funding options: department support and/or a **TÜBİTAK 2209-A** application; earlier department projects obtained this.
- **Compute:** development PCs plus a lab workstation (Koko) already running the headless SITL/Gazebo stack.
- **Cloud AI cost:** the latency sweep uses no LLM, so it costs nothing in API fees. AI-commander experiments cost about US$0.50 per run, so about **US$50–150** covers the planned trials.
- **Test area:** netted indoor space. Tests progress in order: props off → tethered → low altitude at low speed.

5.7 Main risks and mitigations

Risk	Mitigation
Gemstone ecosystem is new; ArduPilot on the R5F cores is less proven	Fixed go/no-go at the end of Fall; fallback is Pixhawk + Raspberry Pi with the same software
Accelerator (TIDL) operator support	Compile the frozen YOLOv8n and OSNet models early, before any training
Optical-flow drift on plain indoor floors	Textured floor mats, AprilTag landing, low speed caps
Re-ID trained on eye-level footage fails from above	Early test on drone-view footage; fall back to attributes and fine-tuning
Indoor flight safety	Netted area, prop guards, RC safety pilot, speed caps, props-off tests first
Wi-Fi instability	Command TTL, hold-by-default, link-loss RTH
API cost and latency	Event-driven queries, low-resolution frames, optional edge tier
Privacy (KVKK)	Mission-scoped profiles, deletion after mission, encrypted logs; human confirms every match

5.8 Project complexity (COMP 4910 CPX checklist)

The jury must agree on at least 2 of the 13 technical factors. HORUS meets these clearly:

- **T1 Distributed system:** cloud, onboard computer, flight controller and operator console.
- **T2 Response time:** real-time control under 0–3 s link delay.
- **T4 Internal processing complexity:** tracking, re-ID, latency compensation, state machine.
- **T10 Concurrency:** threaded MAVLink, perception, executor and event streams.
- **T11 Security:** authentication, tunnel, prompt-injection handling, KVKK.

It also meets these partially:

- **T5 Code reusability:** the same stack runs in SITL and on hardware.
- **T8 Portability:** swappable commander backends and flight computers.

6. Open items before PAF submission (due 9 Oct 2026)

- Team members' ID numbers, names, e-mails and phones for the PAF form; assign members A–D.
- Confirm the hardware budget source (department / TÜBİTAK 2209-A) and whether a Gemstone board is available.
- Book a netted indoor test area on campus.